

```
In [1]: import numpy as np
import pandas as pd

import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings(action = 'ignore')

from IPython.display import Image

for i in [np, pd, mpl, sns]:
    print(i.__name__, i.__version__)
```

```
numpy 1.26.4
pandas 2.1.4
matplotlib 3.8.0
seaborn 0.12.2
```

3-1 머신러닝(Machine Learning)

- 컴퓨터(연산 장치, Machine)로 명시적인 규칙(또는 프로그램) 없이 데이터를 통해 성능을 향상시키는 방법을 연구하는 분야
- 머신러닝 목표: 실제 상황에서 다양한 데이터와 상황에 있어도 모델의 좋은 성능을 유지하도록 학습하는 것 → 일반화(Generalization)

```
In [2]: Image('111.png')
```

Out[2]:

0. 데이터셋 소개

Abalone

[Abalone](#) 데이터셋: 전복의 생육 정보 데이터입니다.

Name	Data Type	Meas.	Description
Sex	nominal		M, F, and I (infant)
Length	continuous	mm	Longest shell measurement
Diameter	continuous	mm	perpendicular to length
Height	continuous	mm	with meat in shell
Whole	continuous	grams	whole abalone
Shucked	continuous	grams	weight of meat
Viscera	continuous	grams	gut weight (after bleeding)
Shell	continuous	grams	after being dried
Rings	integer		+1.5 gives the age in years

```
In [2]: df_abalone = pd.read_csv('data/abalone.csv')
df_abalone.head()
```

```
Out[2]:
```

	Sex	Length	Diameter	Height	Whole	Shucked	Viscera	Shell	Rings
0	M	0.455	0.365	0.095	0.5140	0.2245	0.1010	0.150	15
1	M	0.350	0.265	0.090	0.2255	0.0995	0.0485	0.070	7
2	F	0.530	0.420	0.135	0.6770	0.2565	0.1415	0.210	9
3	M	0.440	0.365	0.125	0.5160	0.2155	0.1140	0.155	10
4	I	0.330	0.255	0.080	0.2050	0.0895	0.0395	0.055	7

머신러닝 모델(Machine Learning Model)

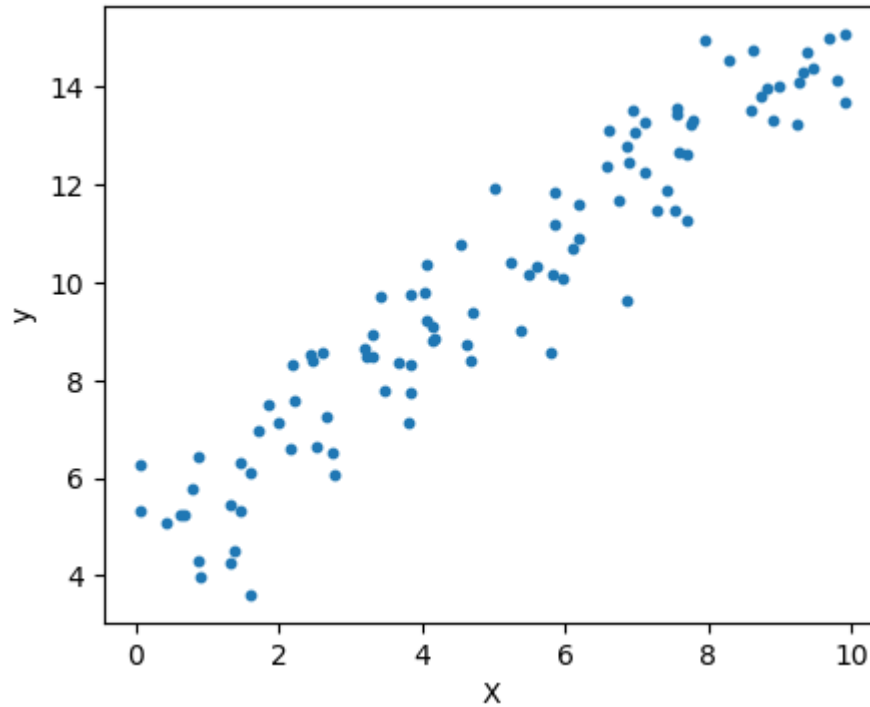
- 데이터에서 추출한 지식(패턴)을 연산 장치(컴퓨터)에 근사화한 객체
- 주어진 데이터의 패턴을 학습하여 예측을 수행함

[Ex.1]

선형 관계를 지니고 있는 데이터를 선형 함수로 근사화해 봅니다.

기울기가 1이고 절편이 5인 직선을 중심으로 형성된 100개의 임의의 데이터 포인트를 생성해봅니다.

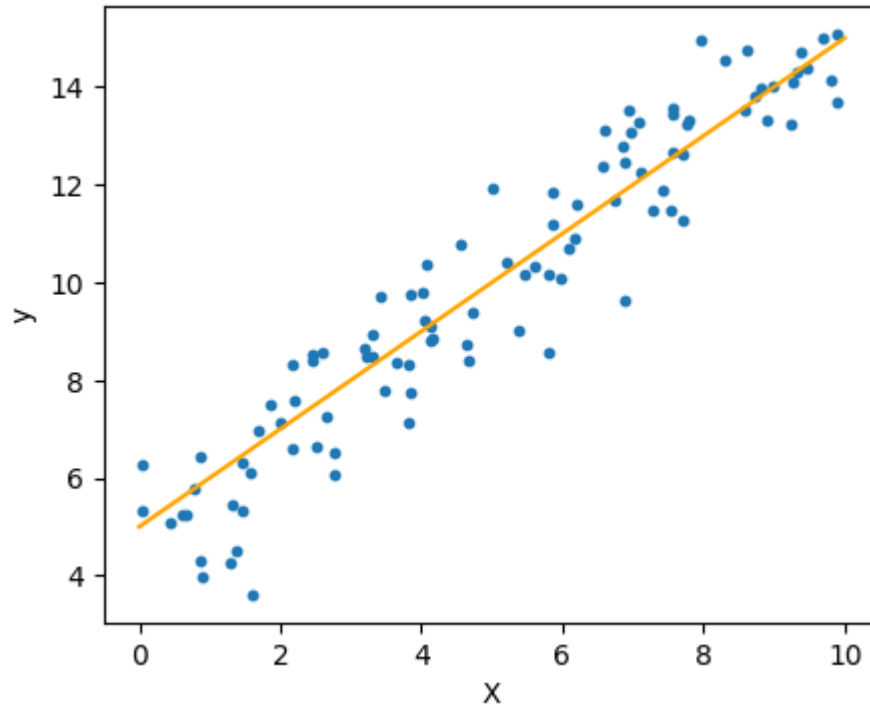
```
In [3]: a, b= 1, 5
X = np.random.random(size=100) * 10
y = X * a + b + np.random.normal(size=100)
plt.figure(figsize=(5, 4))
plt.scatter(X, y, s=10)
plt.gca().set_xlabel('X')
plt.gca().set_ylabel('y')
plt.show()
```



위와 같이 생성된 데이터를 직선으로 근사화 해보면,

기울기가 1이고 절편이 5인 직선이 됩니다. 이를 도식화하면 아래와 같습니다.

```
In [4]: a, b = 1, 5
plt.figure(figsize=(5, 4))
plt.scatter(X, y, s=10)
plt.plot((0, 10), (a * 0 + b, a * 10 + b), c='orange')
plt.gca().set_xlabel('X')
plt.gca().set_ylabel('Y')
plt.show()
```



```
In [3]: Image('112.png')
```

Out[3]: **머신러닝 모델의 분류**

모수적 모델(Parametric Model)

- 주어진 데이터가 특정한 분포를 따른다고 가정한 뒤 예측을 수행합니다.

함수로 데이터의 관계를 나타내고, 데이터를 통해 모수(계수, Coefficient, 가중치: Weight)를 구하는 학습과정을 통해 성능을 향상시킵니다.

선형 회귀 모델, 로지스틱 회귀 모델, 인공신경망

비모수적 모델(Non-parametric Model)

- 데이터 분포에 대한 가정 없이, 주어진 데이터만을 사용하여 예측을 수행합니다.

의사 결정 나무(Decision Tree), K-최근접이웃(K-Nearest Neighbors), ...

손실(Loss)

- 모델링을 통한 근사화에 의해 발생한 실제 데이터와의 오차를

정량화: 클수록 실제 데이터와 차이가 많이 남을 뜻합니다.

In [4]: `Image('113.png')`

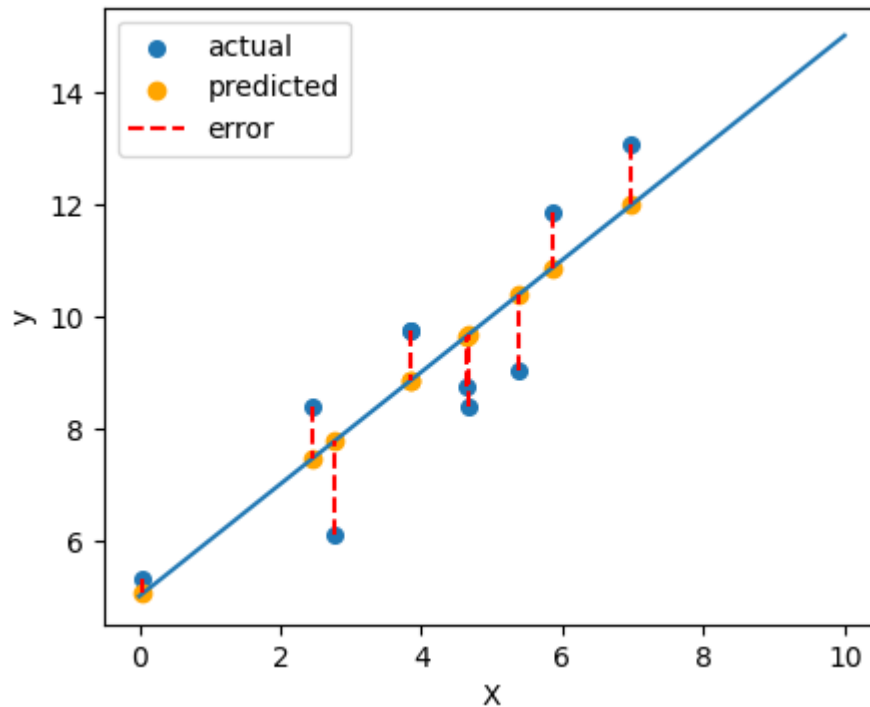
Out[4]: **[Ex.2]**

Ex.1의 데이터에서 임의의 10개의 데이터 포인트를 뽑아 손실(loss)를 구해봅니다.

Absolute error = $|y - \hat{y}|$

Squared error = $(y - \hat{y})^2$

```
In [5]: idx_10 = np.random.choice(np.arange(100), 10)
X2, y2 = X[idx_10], y[idx_10]
plt.figure(figsize=(5, 4))
plt.plot((0, 10), (a * 0 + b, a * 10 + b))
plt.scatter(X2, y2, s=30, label='actual')
plt.scatter(X2, X2 * a + b, color='orange',
            label='predicted')
plt.vlines(X2, y2, X2 * a + b, linestyles='dashed',
            color='red', label='error')
plt.gca().set_xlabel('X')
plt.gca().set_ylabel('y')
plt.legend()
plt.show()
```



```
In [6]: '''
코드는 선형 회귀 모델의 예측 결과를 계산하고
오차를 계산하는 과정을 나타냅니다.
```

1. DataFrame 생성:

```
- `pd.DataFrame({'X': X, 'y': y})`:
```

독립 변수 X 와 종속 변수 y 를 갖는 DataFrame을 생성합니다.

2. 예측값 계산:

```
- `.assign(y_hat = lambda x: X * a + b)`:
```

생성된 DataFrame에 대해, 주어진 회귀 모델의 계수(a 와 b)를 이용하여 예측값 y_hat 을 계산합니다.
이 때, X 는 독립 변수를 나타냅니다.

3. 오차 계산:

```
- `.assign(error = lambda x: x['y'] - x['y_hat'])`:
```

예측값과 실제값의 차이를 계산하여 오차($error$)를 새로운 열로 추가합니다.

4. 절대 오차 및 제곱 오차 계산:

```
- `.assign(absolute_error = lambda x: x['error'].abs(),
          squared_error = lambda x: x['error'] ** 2)`:
```

이전에 계산된 오차를 바탕으로, 절대 오차($absolute_error$)와 제곱 오차($squared_error$)를 계산하여 새로운 열로 추가합니다.

```
- `absolute_error`:
```

각 데이터 포인트에서의 예측값과 실제값 사이의 절대적인 차이를 나타냅니다.

```
- `squared_error`:
```

각 데이터 포인트에서의 예측값과 실제값 사이의 차이를 제곱한 값입니다.

이렇게 계산된 예측값과 오차는 모델의 성능을 평가하고 모델의 개선 여부를 판단하는 데 사용될 수 있습니다.

```
'''
```

```
pd.DataFrame(
    {'X': X, 'y': y}
).assign(
    y_hat = lambda x: X * a + b
).assign(
    error = lambda x: x['y'] - x['y_hat'], # 오차
).assign(
    absolute_error = lambda x: x['error'].abs(),
```

```
squared_error = lambda x: x['error'] ** 2
)
```

Out[6]:

	X	y	y_hat	error	absolute_error	squared_error
0	2.603227	8.562464	7.603227	0.959237	0.959237	0.920135
1	5.474393	10.169723	10.474393	-0.304670	0.304670	0.092824
2	8.894571	13.308228	13.894571	-0.586343	0.586343	0.343798
3	6.093356	10.703443	11.093356	-0.389912	0.389912	0.152032
4	7.531229	11.481228	12.531229	-1.050001	1.050001	1.102502
...	...	...	...	...	...	...
95	9.372568	14.729912	14.372568	0.357344	0.357344	0.127694
96	6.592768	13.098191	11.592768	1.505423	1.505423	2.266298
97	4.026391	9.787115	9.026391	0.760724	0.760724	0.578701
98	4.543819	10.760261	9.543819	1.216442	1.216442	1.479732
99	1.380184	4.523479	6.380184	-1.856705	1.856705	3.447353

100 rows × 6 columns

In [5]: `Image('114.png')`

Out[5]: 학습(Learning, Train)

머신러닝 모델의 손실(Loss)을 줄이는 과정입니다.

지도학습(Supervised Learning)

입력 데이터(Input data)로 대상 데이터(Target data)를 나타낼 때 손실을 줄이는 과정입니다.

대상 데이터에 대한 예측을 출력합니다.

지도학습 모델은 입력 데이터에서 대상 데이터를 예측하기 위한 지식(패턴)을 지니고 있습니다.

대상 데이터의 종류에 따라 구분됩니다.

1. 분류(Classification): 대상 데이터가 범주형

- 모델의 예측값과 실제 클래스 사이의 오분류를 줄이는 방향으로 학습하는 작업입니다.

손실(Loss)

- cross entropy = $-\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^M p_{ij} \log(q_{ij})$,

M 은 클래스의 총 수입니다.

p_{ij} : i 번째 데이터가 j 번째 클래스에 속할 실제 확률(속하는 여부, 0: 속하지 않음, 1: 속함)입니다.

q_{ij} : i 번째 데이터가 j 번째 클래스에 속할 것으로 모델이 예측한 확률입니다.

- binary cross entropy = $-\frac{1}{N} \sum_{i=1}^N (p_i \log(q_i) + (1 - p_i) \log(1 - q_i))$: i 클래스일 확률, y_i : i 클래스 여부

N 은 데이터 포인트의 총 수입니다.

p_i : i 번째 데이터의 클래스입니다. 0: Negative 클래스, 1: Positive 클래스

q_j : i 번째 데이터의 Positive 클래스일 예측 확률입니다.

2. 회귀(Regression): 대상 데이터가 수치형

- 모델의 예측값과 실제 정답값 사이의 차이를 줄이는 방향으로 학습하는 작업입니다.

손실(Loss)

- Mean Squared Error = $\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$

In [6]: `Image('115.png')`

Out[6]: 비지도 학습(Unsupervised Learning)

데이터를 모델로 나타낼 때 손실을 줄이는 과정입니다.

비지도학습 모델은 데이터를 효과적으로 나타내기 위한 지식(패턴)을 지니고 있습니다.

비지도 학습 모델이 나타내는 데이터에 따라 아래와 같이 구분 됩니다.

1. 군집화(Clustering): 주어진 데이터들을 비슷한 그룹으로 묶는 작업입니다.
2. 차원축소(Dimension reduction): 고차원 데이터를 저차원의 데이터로 변환하는 작업입니다.
3. 속성 추출(Feature extraction): 데이터의 복잡한 특성을 간결하고 유용한 형태로 변환하는 작업입니다.
4. 이상치 탐지(Outlier detection): 데이터의 이상치 여부를 확인하고 식별하는 작업입니다.

In [8]: `Image('116.png')`

Out[8]: **지표(Metric)**

- 머신러닝 모델의 성능을 나타내는 기준

분류 지표

이진 분류(Binary-class Classification)

지표	설명
정확도(Accuracy)	$\frac{TP + TN}{TP + FN + TN + FP}$
정밀도(Precision)	$\frac{TP}{TP + FP}$
민감도(Sensitivity), 재현율(Recall)	$\frac{TP}{TP + FN}$
특이도(Specificity)	$\frac{TN}{TN + FP}$
F1-score	$\frac{2 \times Precision \times Recall}{Precision + Recall}$
AUC(Area Under receiver characteristic Curve)	ROC 커브의 하부 면적

예측 \ 실제	Positive	Negative
Positive	TP(True Positive)	FP(False Positive)
Negative	FN(False Negative)	TN(True Negative)

In [9]: `Image('117.png')`

Out[9]: 다중 클래스 분류(Multi-class Classification)

- 정밀도, 재현율, F1-score 같이 Positive 클래스를 기준으로 측정하는 지표는 다중 클래스에서는 Positive 클래스를 정하는 방법과, 여러 개의 Positive 클래스를 사용하여 점수를 집계하는 방법을 지정할 필요가 생깁니다.

1. Positive / Negative 클래스를 지정하는 방법에 따른 구분

- OVR**(One-vs-Rest): Positive 클래스외에 나머지는 Negative 클래스로 지정합니다.
- OVO**(One-vs-One): Positive 클래스와 Negative 클래스를 개별로 지정합니다.

2. Positive/Negative 클래스의 경우가 여러 가지 있다면, 각 클래스별 집계를 내는 방법에 따라구분됩니다.

- Micro**: 개별 클래스에 대한 TP, FP, FN를 산출하여 구합니다., Micro Recall, Micro Precision, ...

$$\text{Micro Recall} = \frac{TP_1 + TP_2 + \dots + TP_n}{TP_1 + TP_2 + \dots + TP_n + FN_1 + FN_2 + \dots + FN_n}$$

$$\text{Micro Precision} = \frac{TP_1 + TP_2 + \dots + TP_n}{TP_1 + TP_2 + \dots + TP_n + FP_1 + FP_2 + \dots + FP_n}$$

- Macro**: 클래스의 비율과 관계없이, 각 클래스별 평균으로 계산: Macro Recall, Macro Precision, ...

$$\text{Macro Recall} = \frac{1}{n} \sum_{i=1}^n \frac{TP_i}{TP_i + FN_i}$$

$$\text{Macro Precision} = \frac{1}{n} \sum_{i=1}^n \frac{TP_i}{TP_i + FP_i}$$

$TP_1, TP_2, \dots, TP_n$:은 n번 클래스의 True Positive

$FN_1, FN_2, \dots, FN_n$:은 n번 클래스의 False Negative

$FP_1, FP_2, \dots, FP_n$:은 n번 클래스의 False Positive

In [10]: Image('118.png')

Out[10]: 회귀 지표

지표	설명
평균제곱오차(MSE, Mean Squared Error)	$\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$
평균절대오차(MAE, Mean Absolute Error)	$\frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i $
평균절대비율오차(MAPE, Mean Absolute Percentage Error)	$\frac{100}{n} \sum_{i=1}^n \left \frac{y_i - \hat{y}_i}{y_i} \right $
R2 점수(R squared score)	$1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$

군집화 지표

지표	설명
중심거리제곱합(Inertia, Sum of Squared error)	$\text{Inertia} = \sum_{i=1}^k \sum_{j=1}^{n_i} x_{ij} - c_i ^2$ k: 군집 수 n_i: C_i의 데이터 포인트 수 x_{ij}: C_i의 j번째 데이터 포인트 c_i: C_i의 중심(평균)입니다
실루엣 점수(Silhouette score)	$S(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$ $S(i)$: 데이터 포인트 i의 실루엣 계수 $a(i)$: 같은 군집 내의 모든 다른 포인트와의 평균 거리 $b(i)$: 가장 가까운 이웃 클러스터에 속한 모든 포인트와의 평균 거리

$$\text{Silhouette Score} = \frac{1}{N} \sum_{i=1}^N S(i)$$

$$\text{Dunn Index} = \frac{\min_{i \neq j} \left(\min_{x \in C_i, y \in C_j} d(x, y) \right)}{\max_k \left(\max_{x, y \in C_k} d(x, y) \right)}$$

던 지표(Dunn index)

C_i, C_j, C_k : 각각 클러스터를 나타냅니다.
 $d(x, y)$: x, y 의 거리

In [11]: `Image('119.png')`

Out[11]: **평가(Test)**

- 머신러닝의 과제의 목표를 효과적으로 나타낼 수 있는 지표를 선택합니다.
- 데이터를 학습 데이터(Train data)와 평가 데이터(Test data)로 구성합니다.
- 머신러닝 모델을 실제로 적용하는 상황을 고려하여 구성합니다.

주식 가격 예측: 주식 가격은 과거의 데이터를 바탕으로 앞으로의 주식 가격을 예측합니다. → 특정 시점을 기준으로 이전은 학습 데이터로 이 후는 평가 데이터로 구성합니다.

신제품 수요 예측: 신제품은 데이터가 축적이 되지 않은 상태입니다. → 평가 데이터는 학습 데이터에 등장하지 않은 제품의 데이터로 구성합니다.

In [12]: `Image('120.png')`

Out[12]: 검증(Validation)

머신러닝을 평가했을 때의 예상 성능을 가늠해보는 과정입니다. → 검증의 결과가 평가의 결과와 유사할 수록 좋은 검증입니다.

검증의 전략: 평가 방법의 특성을 반영하여 검증 방법 을 구성합니다.

검증법

학습 데이터를 검증용 학습데이터와 **검증용 평가데이터(검증 데이터)**를 선택 방법과 검증을 진행 방법하는 따라 구분됩니다.

선택 방법

계층적 선택법(Stratified Selection)

- 분류 데이터셋에서 검증용 학습데이터와 검증용 평가데이터를 구성할 때 대상 데이터(target)의 분포를 양쪽에 동일하게 구성합니다.

그룹화 선택법(Grouped Selection)

- 범주형 데이터를 기준으로 검증용 학습 데이터에는 등장하지 않는 범주를 검증용 평가 데이터에 구성합니다.

시계열 선택법(Time Series Selection)

- 데이터의 시점을 T 시점 기준으로 T 시점 이전을 검증용 학습 데이터에 T 시점 이후를 검증용 평가 데이터로 구성합니다.

In [13]: Image('121.png')

Out[13]: **진행 방법****홀드아웃 검증(Holdout Validation)**

1. 검증용 학습데이터와 검증용 평가데이터로 나눕니다.
2. 검증용 학습데이터로 모델을 학습시키고 검증용 평가 데이터로 성능을 측정합니다.

교차 검증(Cross Validation)

- 데이터를 여러 개의 폴드(겹, fold)로 나누고 각 폴드를 한 번씩 검증 데이터로 나머지는 학습 데이터로 사용하여 검증 결과를 생성합니다.
- 폴드 수만큼 나온 검증 결과를 평균을 내어 성능을 측정합니다.
- 교차 검증은 폴드의 수만큼 학습과 검증이 되기 때문에 데이터가 상대적으로 적거나 변인이 많아 정교한 검증이 요구가 될 경우 채택을 합니다.
- 변인이 많은 상황에서 신뢰도 높은 검증을 하고자 할 경우에 교차검증을 여러 번 진행하기도 합니다.



In [14]: Image('122.png')

Out[14]: **모델 선택**

모델의 선택 과정

1. 과제에 대한 정확한 이해를 바탕으로 검증 방법 정의합니다.
2. 데이터 이해하여 데이터의 특성, 분포, 이상치 결측치들을 고려하여 전처리 과정을 정의합니다.
3. 모델 제작

데이터 전처리 → 속성 공학 → 모델 정의

4. 검증 및 결과 분석하여 모델의 성능이 안 좋은 데이터의 유형을 분석하여 개선 방안을 도출합니다.
5. 2~4 단계를 반복하면서 모델 선별
6. 전체 학습 데이터로 재학습합니다.

[Ex.2]

df_abalone으로 abalone(전복)의 성별(Sex)을 맞추는 머신러닝 모델과제를 위한 평가 데이터를 만듭니다.

평가 요건

평가 데이터는 성비를 맞추어 전체 데이터에서 20%를 무작위로 뽑습니다.

학습데이터는 df_train, 테스트데이터는 df_test로 저장합니다.

```
In [7]: from sklearn.model_selection import train_test_split
df_train, df_test = train_test_split(
    df_abalone, test_size=0.2, stratify=df_abalone['Sex'],
    random_state=123
)
```

[Ex.3]

df_abalone 데이터의 성별(Sex)을 예측하는 머신러닝 모델을 만들어봅니다.

평가 지표는 정확도(Accuracy)입니다.

검증과정을 정의하고 구현해보고,

학습데이터 df_train으로 성별을 구분하는

여러 모델을 자유롭게 만들어 검증을 해봅니다.

모델의 성능을 df_test로 평가합니다.

```
In [8]: '''
위 코드는 분류 모델의 성능을 평가하기 위한
함수들을 정의하고 있습니다.

1. 데이터 준비:
- `y_target = 'Sex'`:
    종속 변수를 설정합니다.
- `X_cols`:
    입력 변수를 정의합니다.
    종속 변수를 제외한 모든 열이 입력 변수로 사용됩니다.

2. 교차 검증(Cross-validation) 설정:
- `cv = StratifiedKFold(n_splits=5, random_state=123,
    shuffle=True)` :
    5-fold 층화(Stratified) 교차검증 Splitter를 설정합니다.
- `n_splits=5`:
    데이터를 5개의 폴드로 나눕니다.
- `random_state=123`:
    재현 가능한 결과를 위해 난수 시드를 설정합니다.
- `shuffle=True`:
    데이터를 섞어서 폴드를 생성합니다.

3. 모델 평가를 위한 함수 `eval_model()` :
- 주어진 분류기(clf)를 사용하여 교차 검증을
    수행하고 모델의 성능을 평가합니다.
- Parameters:
    - `clf`:
```

```

        Classifier - 분류기 인스턴스
    - Returns:
        - 학습 평균(Training Mean),
          학습 표준편차(Training Standard Deviation),
          검증 평균(Test Mean),
          검증 표준편차(Test Standard Deviation)

4. 모델 테스트를 위한 함수 `test_model()` :
    - 주어진 분류기(clf)를 사용하여 테스트 데이터에
      대한 성능을 평가합니다.
    - Parameters:
        - `clf`:
            Classifier - 분류기 인스턴스
    - Returns:
        - 평가 결과 (Accuracy)

이러한 함수들을 사용하여 모델을 교차 검증하고
테스트하는 과정에서 모델의 성능을 평가할 수 있습니다.
'''

from sklearn.model_selection
import StratifiedKFold, ShuffleSplit, cross_validate
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

y_target = 'Sex'
# 입력 변수를 정의합니다.
X_cols = [i for i in df_train.columns if i != y_target]
# 5-fold 층화 교차 검증 Splitter를 설정합니다.
cv = StratifiedKFold(n_splits=5, random_state=123,
                     shuffle=True)

def eval_model(clf):
    """
    모델의 성능을 검증합니다.
    Parameters:
        clf: Classifier
            분류기 인스턴스
    Returns:
        학습 평균, 학습 표준편차, 검증 평균, 검증 표준편차

```

```

"""
result = cross_validate(clf, df_train[X_cols],
                        df_train[y_target],
                        cv=cv, scoring='accuracy',
                        return_train_score=True)

return np.mean(result['train_score']),
       np.std(result['train_score']),
       np.mean(result['test_score']),
       np.std(result['test_score'])

def test_model(clf):
    """
    모델을 평가합니다.
    Parameters:
        clf: Classifier
            분류기 인스턴스
    Returns:
        평가 결과 (Accuracy)
    """
    prd = clf.predict(df_test[X_cols])
    return accuracy_score(df_test['Sex'], prd)

```

In [9]:

```

'''
이 부분은 `make_pipeline` 함수를 사용하여
여러 전처리 단계를 포함하는 파이프라인을 만들고,
그 파이프라인에 로지스틱 회귀(Logistic Regression)
분류기를 추가합니다.
로지스틱 회귀 분류기는 소프트맥스 손실 함수를
사용하여 다중 클래스 분류를 수행합니다.

eval_model(clf_lr):
이 부분은 정의된 `eval_model` 함수를 호출하고,
이 함수에 `clf_lr`을 전달하여 로지스틱 회귀 분류기의
성능을 평가합니다. 함수는 학습 데이터와 검증 데이터에 대한
정확도의 평균과 표준 편차를 반환합니다.
'''

'''
1. 첫 번째 값 (0.5706377489717693):
    학습 데이터의 정확도 평균

```

- 이 값은 학습 데이터에 대한 분류 모델의 평균 정확도를 나타냅니다.
즉, 모델이 학습 데이터에 대해 약 57.06%의 정확도를 보입니다.

2. 두 번째 값 (0.0025117044396221083):

학습 데이터의 정확도 표준 편차

- 이 값은 학습 데이터의 정확도에 대한 표준 편차를 나타냅니다.
즉, 각 교차 검증 폴드에서 얻은 정확도가 이 값의 표준 편차만큼 변동할 수 있습니다.

3. 세 번째 값 (0.5662886559707545):

검증 데이터의 정확도 평균

- 이 값은 검증 데이터에 대한 분류 모델의 평균 정확도를 나타냅니다.
즉, 모델이 검증 데이터에 대해 약 56.63%의 정확도를 보입니다.

4. 네 번째 값 (0.01286315145162258):

검증 데이터의 정확도 표준 편차

- 이 값은 검증 데이터의 정확도에 대한 표준 편차를 나타냅니다.
즉, 각 교차 검증 폴드에서 얻은 정확도가 이 값의 표준 편차만큼 변동할 수 있습니다.

이러한 결과를 통해 모델이 학습 데이터와 검증 데이터 모두에서 비슷한 성능을 보이지만, 일부 폴드에서는 약간의 정확도 차이가 있을 수 있음을 알 수 있습니다.

...

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import mean_squared_error
```

로지스틱 회귀 모델을 만들어 검증해봅니다.

StandardScaler → LogisticRegression

```
clf_lr = make_pipeline(
    StandardScaler(),
    # 다중 클래스 확률을 softmax로 계산토록합니다.
    LogisticRegression(solver='lbfgs',
                       multi_class='multinomial')
)
eval_model(clf_lr)
```

```
Out[9]: (0.5706377489717693,
         0.0025117044396221083,
         0.5662886559707545,
         0.01286315145162258)
```

```
In [10]: '''
- clf_lr.fit(df_train[X_cols], df_train[y_target]):
    이 부분은 입력된 분류기(`clf`)를 사용하여
    학습 데이터(`df_train[X_cols]`, `df_train[y_target]`)에 대해
    모델을 학습합니다. `fit` 메서드를 사용하여 모델을 학습시킵니다.
- test_model(clf_lr):
    이 부분은 정의된 `test_model` 함수를 호출하고,
    이 함수에 `clf_lr`을 전달하여 로지스틱 회귀 분류기의 성능을
    테스트 데이터를 통해 평가합니다.
    함수는 테스트 데이터에 대한 정확도를 반환합니다.
즉, `test_model(clf_lr)` 코드는 학습된 분류기를 사용하여
테스트 데이터를 예측하고, 그 예측 결과의 정확도를 반환합니다.
'''

# 로지스틱 회귀 모델을 전체 학습 데이터로
# 학습하고, 평가해봅니다.
clf_lr.fit(df_train[X_cols], df_train[y_target])
test_model(clf_lr)
```

```
Out[10]: 0.5299043062200957
```

```
In [11]: '''
코드는 RandomForestClassifier를 사용하여
랜덤 포레스트 모델을 생성하고,
그 모델의 성능을 평가하는 것을 목적으로 합니다.

1. RandomForestClassifier 생성:
- RandomForestClassifier 클래스를 사용하여
  랜덤 포레스트 모델을 생성합니다.
- `n_estimators` 매개변수는 생성할 트리의
  개수를 지정합니다. 여기서는 300개의 트리를 생성합니다.
- `max_depth` 매개변수는 각 트리의 최대 깊이를 제한합니다.
  여기서는 각 트리의 최대 깊이를 7로 제한합니다.
- `random_state` 매개변수는 랜덤 시드를 설정하여
  모델의 재현성을 보장합니다. 여기서는 123으로 설정합니다.
'''
```

2. eval_model 함수 호출:

- eval_model 함수를 호출하여 모델의 성능을 검증합니다.
- eval_model 함수는 모델을 입력으로 받아 학습 데이터와 검증 데이터에 대한 정확도의 평균과 표준 편차를 계산하여 반환합니다.

3. 결과 해석:

- 반환된 결과는 학습 데이터와 검증 데이터에 대한 정확도의 평균과 표준 편차입니다.
- 이를 통해 랜덤 포레스트 모델이 학습 데이터와 검증 데이터 모두에서 어느 정도의 성능을 보이는지, 그리고 모델의 일관성을 판단할 수 있습니다.
- 평균 정확도가 높고 표준 편차가 낮을수록 모델이 데이터에 잘 일반화되었음을 의미합니다.

...

```
from sklearn.ensemble import RandomForestClassifier
```

```
# Random Forest 모델을 만들어 검증해봅니다.
```

```
clf_rf = RandomForestClassifier(n_estimators=300,  
                               max_depth=7,  
                               random_state=123)
```

```
eval_model(clf_rf)
```

```
Out[11]: (0.7624965151051286,  
         0.005915177388537954,  
         0.5626882214319154,  
         0.017037374520945774)
```

In [12]:

'''

코드는 RandomForestClassifier를 사용하여 학습된 모델을 테스트 데이터로 평가하는 과정을 수행합니다.

1. RandomForestClassifier 학습:

- `clf\_rf.fit(df\_train[X\_cols], df\_train[y\_target])`
코드는 RandomForestClassifier 모델(`clf\_rf`)을 사용하여 학습 데이터(`df\_train[X\_cols]`)를 학습합니다.
- `df\_train[X\_cols]`는
특성(feature) 데이터(독립 변수)를 나타냅니다.
- `df\_train[y\_target]`는

```

        타겟(target) 데이터(종속 변수)를 나타냅니다.
2. test_model 함수 호출:
    - `test_model(clf_rf)` 코드는 학습된
      RandomForestClassifier 모델을
      테스트 데이터로 평가합니다.
    - `test_model` 함수는 입력된 분류기(`clf_rf`)를
      사용하여 테스트 데이터를 예측하고,
      그 예측 결과의 정확도를 반환합니다.
3. 결과 해석:
    - 반환된 값은 모델이 테스트 데이터에서 얻은
      정확도를 나타냅니다.
    - 이를 통해 모델이 학습 데이터에서
      잘 동작했는지 뿐만 아니라,
      새로운 데이터에 대해서도
      얼마나 잘 일반화되는지를 평가할 수 있습니다.
    - 반환된 정확도가 높을수록 모델이
      테스트 데이터에서 잘 수행되었음을 의미합니다.
'''

# Random Forest 모델을 전체 학습 데이터로
# 학습하고, 평가해봅니다.
clf_rf.fit(df_train[X_cols], df_train[y_target])
test_model(clf_rf)

```

Out[12]: 0.5430622009569378

```

In [13]: '''
코드는 Voting Classifier를 생성하고,
이를 통해 Logistic Regression과
Random Forest 두 모델의 결합을 통해
모델의 성능을 평가하는 것을 목적으로 합니다.

1. Voting Classifier 생성:
    - VotingClassifier 클래스를 사용하여
      Voting Classifier 모델을 생성합니다.
    - VotingClassifier는 여러 개의 분류기를 결합하여
      하나의 모델을 구성하는 앙상블 방법 중 하나입니다.
    - 첫 번째 인자는 리스트 형태로, 각각의 튜플은
      (분류기 이름, 분류기 인스턴스)의 형태를 가집니다.
      여기서는 Logistic Regression과 Random Forest

```

```

두 개의 분류기를 사용합니다.
- voting 매개변수는 예측을 결합하는 방법을 지정합니다.
'soft'로 설정하여 각 분류기의 예측에 대한
확률 평균을 사용하여 최종 예측을 수행합니다.
2. eval_model 함수 호출:
- eval_model 함수를 호출하여
Voting Classifier의 성능을 평가합니다.
- eval_model 함수는 모델을 입력으로 받아
학습 데이터와 검증 데이터에 대한 정확도의
평균과 표준 편차를 계산하여 반환합니다.
3. 결과 해석:
- 반환된 결과는 Voting Classifier의 학습 데이터와
검증 데이터에 대한 정확도의 평균과 표준 편차입니다.
- 이를 통해 Voting Classifier가 어떻게 작동하는지,
개별 모델들의 성능을 어떻게 결합하여
최종 예측을 수행하는지를 평가할 수 있습니다.
- Voting Classifier는 개별 모델들의 예측을 결합하여
보다 안정적이고 강력한 예측을 제공할 수 있습니다.
'''

```

```

from sklearn.ensemble import VotingClassifier

# Voting Classifier를 clf_lr(Logistic Regression)과
# clf_rf(Random Forest)로 구성합니다.
clf_vt = VotingClassifier([
    ('lr', clf_lr),
    ('rf', clf_rf)
], voting='soft')
eval_model(clf_vt)

```

```

Out[13]: (0.6765942879142044,
          0.002459365555234087,
          0.5653833011626678,
          0.01151139827582167)

```

```

In [14]: '''
코드는 Voting Classifier 모델을 학습하고,
그 학습된 모델을 테스트 데이터로 평가한다.

1. Voting Classifier 학습:
- `clf_vt.fit(df_train[X_cols], df_train[y_target])`

```

```

코드는 Voting Classifier 모델(`clf_vt`)을 사용하여
학습 데이터(`df_train[X_cols]`)를 학습합니다.
- 학습 데이터의 특성 데이터(`df_train[X_cols]`)와
타겟 데이터(`df_train[y_target]`)를 사용하여
모델을 학습시킵니다.

2. test_model 함수 호출:
- `test_model(clf_vt)` 코드는 학습된
Voting Classifier 모델을 테스트 데이터로 평가합니다.
- `test_model` 함수는 입력된 분류기(`clf_vt`)를 사용하여
테스트 데이터를 예측하고,
그 예측 결과의 정확도를 반환합니다.

3. 결과 해석:
- 반환된 값은 Voting Classifier 모델이
테스트 데이터에서 얻은 정확도를 나타냅니다.
- 이를 통해 Voting Classifier 모델이 학습 데이터에서
얼마나 잘 동작하고, 새로운 데이터에 대해서도
얼마나 잘 일반화되는지를 평가할 수 있습니다.
- 반환된 정확도가 높을수록 Voting Classifier 모델이
테스트 데이터에서 잘 수행되었음을 의미합니다.
...

# Voting Classifier를 전체 학습 데이터로 학습하고, 평가합니다.
clf_vt.fit(df_train[X_cols], df_train[y_target])
test_model(clf_vt)

```

Out[14]: 0.5358851674641149

[Ex.4]

df_abalone으로 abalone(전복)의 연령(Rings)을 맞추는

머신러닝 모델과제를 위한 평가 데이터를 만듭니다.

평가 요건

평가 데이터는 성별(Sex)이 I(Infant)인 것으로만 구성하고,

학습 데이터는 성별이 I가 아닌 것으로 구성합니다.

학습데이터는 df_train2, 테스트데이터는 df_test2로 저장합니다.

```
In [15]: df_train2 = df_abalone.loc[df_abalone['Sex'] != 'I']
df_test2 = df_abalone.loc[df_abalone['Sex'] == 'I']
```

[Ex.5]

학습데이터 df_train2으로 연령(Rings)을 맞추는 모델을 자유롭게 만들어봅니다.

평가 지표는 평균제곱오차(MSE, Mean Squared Error)입니다.

검증과정을 정의하고 구현해보고,

학습데이터 df_train으로 연령(Rings)을 구분하는

여러 모델을 자유롭게 만들어 검증을 해봅니다.

모델의 성능을 df_test로 평가합니다.

```
In [16]: '''
코드는 GroupKFold를 사용하여 그룹별로 데이터를
분할하는 방법을 설정하는 것을 목적으로 합니다.

1. GroupKFold 설정:
- `from sklearn.model_selection import GroupKFold`를
  사용하여 GroupKFold를 import합니다.
  GroupKFold는 그룹별 교차 검증을 수행할 때 사용됩니다.
- `n_splits` 매개변수는 데이터를 몇 개의 fold로
  나눌지를 결정합니다. 여기서는 2개의 fold로 나눕니다.
- `cv2` 변수에 GroupKFold 객체를 생성하고,
  이 객체를 사용하여 그룹별로 데이터를
  분할할 수 있도록 준비합니다.
```

2. 그룹 설정:

- `grp` 변수에는 그룹을 식별할 수 있는 열의 이름을 지정합니다.
이 경우 'Sex' 열을 사용하여 데이터를 그룹화합니다.
- 예를 들어, 'Sex' 열이 'male'과 'female' 값으로 이루어져 있다면, 각각의 성별에 해당하는 데이터가 서로 다른 그룹으로 인식됩니다.

3. 타겟 및 특성 데이터 설정:

- `y\_target2` 변수에는 타겟 데이터(종속 변수)의 열 이름을 지정합니다.
이 경우 'Rings' 열을 타겟 데이터로 설정합니다.
- `X\_cols2` 변수에는 모델 학습에 사용할 특성 데이터(독립 변수)의 열 이름을 지정합니다.
여기서는 'Rings' 열과 'Sex' 열을 제외한 나머지 열을 특성 데이터로 설정합니다.
- 이렇게 설정된 특성 데이터는 모델 학습에 사용되며, 각 샘플의 특성을 설명하는 데 사용됩니다.

4. 그룹별 교차 검증 준비:

- GroupKFold를 사용하여 데이터를 그룹별로 분할할 수 있도록 준비합니다.
이를 위해 데이터를 특정 그룹으로 분할하는 방법을 설정합니다.
- 설정된 GroupKFold 객체를 나중에 모델의 교차검증에 적용하여, 그룹별로 데이터를 분할하고 모델의 성능을 평가할 수 있습니다.

'''

```
from sklearn.model_selection import GroupKFold
y_target2 = 'Rings'
grp = 'Sex'
X_cols2 = [i for i in df_train2.columns
            if i != y_target2 and i != grp]
# 등장하지 않는 성별이 Test에 나오므로,
# 검증시 성별을 달리하여 검증 train과 검증 test로
# 나뉘도록 하기 위해 GroupKFold를 씁니다.
cv2 = GroupKFold(n_splits=2)
```

In []:

```
'''
코드는 그룹화된 교차 검증을 통해
```

회귀 모델의 성능을 검증하고, 테스트 데이터를 사용하여 회귀 모델의 성능을 평가하는 두 가지 함수를 정의합니다.

1. eval_model2 함수:

- 그룹화된 교차 검증을 통해 회귀 모델의 성능을 평가합니다.
- ``reg``를 회귀 모델로 간주하고, 이 함수에 입력으로 전달합니다.
- ``cross_validate`` 함수를 사용하여 그룹화된 교차 검증을 수행합니다.
- ``cv=cv2``를 통해 그룹 정보가 포함된 `GroupKFold` 객체(``cv2``)를 사용하여 교차 검증을 수행합니다.
- ``groups=df_train2[grp]``를 통해 각 샘플이 속한 그룹(이 경우 성별)을 기준으로 데이터를 분할합니다.
- 성능 평가 지표로는 평균제곱오차(``scoring='neg_mean_squared_error'``)를 사용합니다.
- ``return_train_score=True``를 통해 학습 데이터의 점수도 반환합니다.
- 반환값으로는 학습 데이터와 검증 데이터에 대한 평균 제곱 오차의 평균과 표준 편차를 계산하여 반환합니다.

2. test_model2 함수:

- 테스트 데이터를 사용하여 회귀 모델의 성능을 평가합니다.
- ``reg``를 회귀 모델로 간주하고, 이 함수에 입력으로 전달합니다.
- 모델을 사용하여 테스트 데이터를 예측하고, 예측 결과와 실제 타겟 값을 사용하여 평균제곱오차를 계산합니다.
- 평균 제곱 오차를 반환합니다.

이러한 함수들을 사용하여 모델의 성능을 평가할 수 있습니다.
`eval_model2` 함수는 교차 검증을 통해 모델의 일반화 성능을 평가하고,
`test_model2` 함수는 모델의 성능을 테스트 데이터를 사용하여 평가합니다.
 ...

```
def eval_model2(reg):
    """
    그룹화 교차 검증을 통한 모델의 성능을 검증합니다.
    Parameters:
        clf: Classifier
            분류기 인스턴스
```

```

Returns:
    학습 평균, 학습 표준편차, 검증 평균, 검증 표준편차
    """
    # groups=df_train2[grp] grp(성별)을
    # 기준으로 나뉘도록 설정합니다.
    result = cross_validate(
        reg, df_train2[X_cols2], df_train2[y_target2],
        cv=cv2, groups=df_train2[grp],
        scoring='neg_mean_squared_error',
        return_train_score=True
    )
    return np.mean(result['train_score']),
           np.std(result['train_score']),
           np.mean(result['test_score']),
           np.std(result['test_score'])

def test_model2(reg):
    """
    모델을 평가합니다.
    Parameters:
        clf: Classifier
            분류기 인스턴스
    Returns:
        평가 결과 (Accuracy)
    """
    prd = reg.predict(df_test2[X_cols2])
    return mean_squared_error(df_test2['Rings'], prd)

```

In [17]: '''
 코드는 선형 회귀(Linear Regression) 모델을
 평가하는 과정을 나타내고 있습니다.

1. Linear Regression 모델 생성:
 - LinearRegression 클래스를 사용하여
선형 회귀 모델을 생성합니다.
 - make_pipeline 함수를 사용하여 StandardScaler를 통해
데이터를 표준화한 후에 LinearRegression 모델을
적용하는 pipeline을 생성합니다.
 - StandardScaler는 각 특성(feature)을 평균이 0이고
표준편차가 1이 되도록 변환하여

데이터의 스케일을 조정합니다.

- `LinearRegression`은 선형 회귀 모델을 생성합니다.

2. `eval_model2` 함수 호출:

- `eval_model2` 함수를 호출하여 모델의 성능을 평가합니다.
- 이 함수는 모델을 입력으로 받아서 그룹화된 교차 검증을 수행하고, 그 결과를 반환합니다.
- `reg_lr`, 즉 `Linear Regression` 모델이 `eval_model2` 함수에 입력으로 전달됩니다.
- `eval_model2` 함수 내부에서는 `cross_validate` 함수를 사용하여 교차 검증을 수행하고, 각 폴드에 대한 평균 제곱 오차를 계산합니다.
- 이렇게 계산된 학습 및 검증 평균 제곱 오차의 평균과 표준 편차가 반환됩니다.

따라서 코드는 선형 회귀 모델을 평가하기 위해 그룹화된 교차 검증을 수행하고, 해당 모델의 성능을 측정하는 과정을 나타냅니다.

```
'''
```

```
from sklearn.linear_model import LinearRegression
```

```
reg_lr = make_pipeline(
    StandardScaler(),
    LinearRegression()
)
eval_model2(reg_lr)
```

```
Out[17]: (-5.658948736875659,
          0.5301771902953769,
          -5.932594716298198,
          0.5366537551139801)
```

```
In [18]: '''
```

코드는 선형 회귀(`Linear Regression`) 모델을 학습하고, 그 학습된 모델을 테스트 데이터로 평가하는 과정을 나타내고 있습니다.

1. 선형 회귀 모델 학습:

- ``reg_lr.fit(df_train2[X_cols2], df_train2[y_target2])`` 코드는 `Linear Regression` 모델(``reg_lr``)을 사용하여

```

        학습 데이터(`df_train2[X_cols2]`)를 학습합니다.
    - 학습 데이터의 특성 데이터(`df_train2[X_cols2]`)와
      타겟 데이터(`df_train2[y_target2]`)를 사용하여
      모델을 학습시킵니다.

2. test_model2 함수 호출:
    - `test_model2(reg_lr)` 코드는 학습된
      Linear Regression 모델을 테스트 데이터로 평가합니다.
    - 이 함수는 입력된 회귀 모델(`reg_lr`)을 사용하여
      테스트 데이터를 예측하고, 그 예측 결과와
      실제 타겟 값 간의 평균 제곱 오차를 계산합니다.
    - 이 평균 제곱 오차를 반환합니다.

따라서 코드는 선형 회귀 모델을 학습하고,
해당 모델의 성능을 테스트 데이터를 사용하여
평가하는 과정을 나타냅니다.
'''

reg_lr.fit(df_train2[X_cols2],
           df_train2[y_target2])
test_model2(reg_lr)

```

Out[18]: 4.609174708960943

```

In [19]: '''
코드는 랜덤 포레스트 회귀(Random Forest Regression)
모델을 평가하는 과정을 나타내고 있습니다.

1. Random Forest Regression 모델 생성:
    - RandomForestRegressor 클래스를 사용하여
      랜덤 포레스트 회귀 모델을 생성합니다.
    - `n_estimators` 매개변수는 생성할 트리의 개수를 지정합니다.
      여기서는 300개의 트리를 생성합니다.
    - `max_depth` 매개변수는 각 트리의 최대 깊이를 제한합니다.
      여기서는 각 트리의 최대 깊이를 7로 제한합니다.
    - `random_state` 매개변수는 랜덤 시드를 설정하여
      모델의 재현성을 보장합니다.

2. eval_model2 함수 호출:
    - eval_model2 함수를 호출하여 모델의 성능을 평가합니다.

```

- 이 함수는 모델을 입력으로 받아서 그룹화된 교차 검증을 수행하고, 그 결과를 반환합니다.
- `reg_rf`, 즉 Random Forest Regression 모델이 `eval_model2` 함수에 입력으로 전달됩니다.
- `eval_model2` 함수 내부에서는 `cross_validate` 함수를 사용하여 교차 검증을 수행하고, 각 폴드에 대한 평균 제곱 오차를 계산합니다.
- 이렇게 계산된 학습 및 검증 평균 제곱 오차의 평균과 표준 편차가 반환됩니다.

따라서 코드는 랜덤 포레스트 회귀 모델을 평가하기 위해 그룹화된 교차 검증을 수행하고, 해당 모델의 성능을 측정하는 과정을 나타냅니다.

```
'''
```

```
from sklearn.ensemble import RandomForestRegressor

reg_rf = RandomForestRegressor(n_estimators=300,
                              max_depth=7,
                              random_state=123)

eval_model2(reg_rf)
```

```
Out[19]: (-3.0299508606178334,
          0.43225112523057496,
          -5.746277817835207,
          0.6302849459652262)
```

```
In [20]: '''
코드는 랜덤 포레스트 회귀(Random Forest Regression) 모델을
학습하고, 그 학습된 모델을 테스트 데이터로
평가하는 과정을 나타내고 있습니다.
```

1. 랜덤 포레스트 회귀 모델 학습:

- `reg_rf.fit(df_train2[X_cols2], df_train2[y_target2])``
코드는 랜덤 포레스트 회귀 모델(`reg_rf``)을 사용하여 학습 데이터(`df_train2[X_cols2]``)를 학습합니다.
- 학습 데이터의 특성 데이터(`df_train2[X_cols2]``)와 타겟 데이터(`df_train2[y_target2]``)를 사용하여 모델을 학습시킵니다.

2. `test_model2` 함수 호출:

- ``test_model2(reg_rf)`` 코드는 학습된 랜덤 포레스트 회귀 모델을 테스트 데이터로 평가합니다.
- 이 함수는 입력된 회귀 모델(``reg_rf``)을 사용하여 테스트 데이터를 예측하고, 그 예측 결과와 실제 타겟 값 간의 평균 제곱 오차를 계산합니다.
- 이 평균 제곱 오차를 반환합니다.

따라서 코드는 랜덤 포레스트 회귀 모델을 학습하고, 해당 모델의 성능을 테스트 데이터를 사용하여 평가하는 과정을 나타냅니다.

```
'''
```

```
reg_rf.fit(df_train2[X_cols2],
           df_train2[y_target2])
test_model2(reg_rf)
```

Out[20]: 4.044116723974344

In [21]:

```
'''
```

코드는 **Voting Regressor**를 사용하여 선형 회귀 모델과 랜덤 포레스트 회귀 모델의 결합을 통해 모델의 성능을 평가하는 것을 목적으로 합니다.

1. Voting Regressor 생성:

- **VotingRegressor** 클래스를 사용하여 **Voting Regressor** 모델을 생성합니다.
- ``('lr', reg_lr)``과 ``('rf', reg_rf)``는 각각 선형 회귀 모델(``reg_lr``)과 랜덤 포레스트 회귀 모델(``reg_rf``)을 지칭하는 튜플입니다.
- **VotingRegressor**는 여러 개의 회귀 모델을 결합하여 하나의 모델을 구성하는 앙상블 방법 중 하나입니다.

2. eval_model2 함수 호출:

- `eval_model2` 함수를 호출하여 **Voting Regressor**의 성능을 평가합니다.
- 이 함수는 모델을 입력으로 받아 학습 데이터와 검증 데이터에 대한 평균 제곱 오차의 평균과 표준 편차를 계산하여 반환합니다.
- `reg_vt`, 즉 **Voting Regressor** 모델이 `eval_model2` 함수에 입력으로 전달됩니다.

- `eval_model2` 함수 내부에서는 `cross_validate` 함수를 사용하여 그룹화된 교차 검증을 수행하고, 각 폴드에 대한 평균 제곱 오차를 계산합니다.
- 이렇게 계산된 학습 및 검증 평균 제곱 오차의 평균과 표준 편차가 반환됩니다.

따라서 코드는 `Voting Regressor`를 사용하여 선형 회귀 모델과 랜덤 포레스트 회귀 모델의 결합을 통해 모델의 성능을 평가하는 과정을 나타냅니다.

```
'''
```

```
from sklearn.ensemble import VotingRegressor
```

```
reg_vt = VotingRegressor([
    ('lr', reg_lr),
    ('rf', reg_rf)
])
eval_model2(reg_vt)
```

```
Out[21]: (-4.030321012231051,
          0.4859813965250255,
          -5.510750554251789,
          0.5910460769415837)
```

```
In [22]: '''
코드는 Voting Regressor 모델을 학습하고,
그 학습된 모델을 테스트 데이터로 평가하는
과정을 나타내고 있습니다.
```

1. Voting Regressor 모델 학습:

- ``reg_vt.fit(df_train2[X_cols2], df_train2[y_target2])``
코드는 `Voting Regressor` 모델(``reg_vt``)을 사용하여 학습 데이터(``df_train2[X_cols2]``)를 학습합니다.
- 학습 데이터의 특성 데이터(``df_train2[X_cols2]``)와 타겟 데이터(``df_train2[y_target2]``)를 사용하여 모델을 학습시킵니다.

2. test_model2 함수 호출:

- ``test_model2(reg_vt)`` 코드는 학습된 `Voting Regressor` 모델을 테스트 데이터로 평가합니다.
- 이 함수는 입력된 회귀 모델(``reg_vt``)을 사용하여

테스트 데이터를 예측하고, 그 예측 결과와 실제 타겟 값 간의 평균 제곱 오차를 계산합니다.
- 이 평균 제곱 오차를 반환합니다.

따라서 코드는 **Voting Regressor** 모델을 학습하고, 해당 모델의 성능을 테스트 데이터를 사용하여 평가하는 과정을 나타냅니다.
...

```
reg_vt.fit(df_train2[X_cols2], df_train2[y_target2])  
test_model2(reg_vt)
```

Out[22]: 4.0508805826831145

머신러닝의 과정

1. 문제 정의
2. 데이터 수집
3. 문제 분석
4. 모델 선택
5. 모델 평가
6. 평가 결과 분석
7. 개선점 도출 후 1~6 단계로 이동

In []: